ALUMNICONNECT

RECOMMENDATION DESIGN

Scoring criteria | Filtering | Ranking | Recommendation explanations

GRAPH DISCOVERY	TRANSPARENT SCORING	MYSQL VALIDATION
Neo4j paths and evidence	Configurable weighted dimensions	Eligibility and capacity remain authoritative

DBMS Mini Project - Alumni-Student Mentorship and Mock Interview Platform

Implementation-oriented specification | Version 1.0 | 2 November 2026

Prepared from the supplied DBMS Project Blueprint and companion AlumniConnect design documents

Document Control and Reading Guide

Item	Specification
Document purpose	Define how AlumniConnect discovers, filters, scores, ranks and explains mentor candidates.
Primary audience	Project team, DBMS designer, backend developers, graph modeller, testers and evaluators.
Product scope	Student / Mentee mentor discovery and recommendation, including history and selection tracking.
Application architecture	Next.js + TypeScript frontend; FastAPI modular monolith; MySQL 8.x / InnoDB, Redis and Neo4j.
Status	Design specification. Scoring weights and heuristic mappings below are proposed starting values.
Source of truth	MySQL owns authoritative identities, profile attributes, capacity, ratings and recommendation history.
Version	1.0 - 2 November 2026

Requirement provenance labels

Label	Meaning
[B] Blueprint-aligned	Explicitly established in the supplied project blueprint or companion architecture.
[PRD] Required behaviour	Specified as a MUST / SHOULD product requirement in the supplied Product Requirements Document.
[R] Recommended design	A concrete scoring, filtering or implementation policy proposed here; confirm with the team.
[P2] Phase 2	Optional extension; not required to deliver the core graph-based recommendation system.

Important distinction

The supplied documents define the recommendation dimensions and require configurable weights, but they do not prescribe numeric weights or a final scoring formula. The 30 / 20 / 15 / 20 / 15 weighting and the detailed formulas in this document are proposed implementation defaults, not pre-approved source requirements.

Contents

Section	Coverage
1-3	Purpose, requirement baseline, architecture and design principles
4-5	Neo4j graph model and end-to-end recommendation pipeline
6	Filtering model: hard eligibility, user filters and soft ranking signals
7-9	Scoring model, formulas, worked example and score interpretation
10-11	Explanation design, recommendation history and relational storage
12-15	Cypher, MySQL validation, Redis caching and cross-database consistency
16-18	Backend/API responsibilities, failure handling, privacy and responsible ranking
19-21	Evaluation, acceptance tests, implementation roadmap and DBMS mapping
Appendices	Open decisions, glossary and source basis

1. Executive Summary and Design Position

AlumniConnect should recommend alumni mentors by combining graph-based relationship discovery with transparent, configurable scoring and authoritative MySQL eligibility checks. The student should be able to see not only who was recommended, but also which skills, domains, experience and career signals contributed to the ranking.

1.1 Recommendation feature goals

- Identify potentially relevant alumni through Neo4j paths connecting Student, Mentor, Skill, Domain, Company and Department nodes. [B]
- Evaluate candidates across skill similarity, domain relevance, relevant experience, career relevance and mentor quality. [PRD FR-09]
- Check current account status, discoverability, capacity and other eligibility conditions in MySQL before a candidate reaches the final list. [PRD FR-09]
- Display understandable reasons such as shared skills, matching domains, relevant experience and prior mentoring outcomes. [PRD FR-10]
- Persist each recommendation run and its candidate scores so the team can compare changes over time and link a selected candidate to a later mentorship request. [PRD FR-10]

1.2 Recommended v1 design

Concern	Recommended decision
Candidate generation	Neo4j graph traversal; do not rely only on a large set of relational joins.
Eligibility	Hard filters in MySQL; a mentor at capacity is excluded rather than merely given a lower score.
Ranking	Weighted score from five configurable dimensions; default weights are proposed in Section 7.
Explanation	Generate reason codes and human-readable statements from the exact matched graph and SQL evidence.
History	Persist Recommendation Run and Recommended Mentor rows; retain the criteria version and input snapshot.
Performance	Redis cache-aside for recent recommendation results; invalidate when relevant source data changes.
Advanced semantic matching	Optional Redis Stack vector search in Phase 2, combined with graph retrieval and SQL validation.

Not a black-box AI decision

For the first release, use a deterministic graph + weighted-rule engine. This is easier to explain in a DBMS viva, easier to test, and consistent with the blueprint. A recommendation is a candidate suggestion, not an automatic mentorship allocation.

1.3 What a recommendation is - and is not

Concept	Meaning
Recommendation	A ranked suggestion that a mentor may fit the student profile and goal.
Mentorship request	The student's explicit proposal to a selected mentor; it enters Pending.
Mentor allocation	The accepted student-mentor relationship created when the mentor accepts.
Booking	A planned appointment against one published availability slot.
Mentorship session	The actual offline meeting and its recorded outcome.

2. Requirement Baseline and Source Alignment

The recommendation design is grounded in the supplied project blueprint, Product Requirements Document (PRD), Application Architecture, Conceptual ER Diagram and Workflow / State Diagrams. The recommendation-specific source requirements are concentrated in PRD FR-08 to FR-10 and the blueprint Neo4j recommendation section.

Source	What it establishes	Design consequence
PRD FR-08 [MUST]	Mentor directory search by skills, domains, company and professional experience; protect private information.	Support explicit filters and return only permitted mentor-card fields.

Source	What it establishes	Design consequence
PRD FR-09 [MUST]	Neo4j graph traversal; dimensions include skills, domains, experience, career relevance, mentor quality and capacity; weights configurable.	Use graph candidate generation, configurable scoring, and MySQL eligibility validation.
PRD FR-10 [SHOULD]	Explain recommendations and retain recommendation runs and candidate results.	Persist criteria version, score components, explanation and selection outcome.
Blueprint §5.1-5.5	Graph labels / relationships, multiple matching dimensions, pipeline, explainability and MySQL > Neo4j.	Model evidence as traversable relationships; show reasons; exclude mentors who fail current MySQL checks.
Architecture §7.2-8	Neo4j as projection, Redis as cache; transactional outbox updates the graph and invalidates caches.	Never treat Redis or Neo4j as authoritative; use eventual projection sync.
ER Diagram pp. 10-14	Recommendation Run and Recommended Mentor entities; rank, score, explanation and selected flag.	Use one run per calculation and one candidate row per result.
Blueprint §9 / PRD success indicators	History should answer what was recommended, what score was assigned, what was selected and whether sessions followed.	Support recommendation funnel and outcome evaluation.

2.1 Explicitly required vs proposed choices

Already established	Proposed in this document
Graph-based mentor candidate discovery.	Specific Cypher query patterns and graph indexes.
Five ranking dimensions plus capacity / eligibility.	Default weights: 30 / 20 / 15 / 20 / 15.
Weights must be configurable.	Criteria versioning and stored weight snapshots.
MySQL validates active eligibility and capacity.	Exact eligibility checklist and SQL template.
Explanations and recommendation history are desired.	Explanation JSON schema, reason-code catalogue and run retention design.
Redis can cache recommendation results.	Example cache key and an initial TTL / invalidation policy.
Vector search is a later optional extension.	Phase 2 design and rollout conditions.

Source gap made explicit

The blueprint does not set numerical weights, exact thresholds, minimum review counts, tie-break rules, a candidate pool size or cache TTL. These values are presented as tunable project defaults and should be recorded in the criteria configuration rather than embedded permanently in code.

3. Architecture and Component Responsibilities

The recommendation feature fits the project's modular monolith: one FastAPI backend coordinates the discovery module and the specialised data stores. The browser never talks directly to MySQL, Redis or Neo4j.

Component	Responsibility in recommendations	Must not do
Student portal (Next.js)	Collect discovery criteria; show mentor cards, score breakdown and explanations; record the selected mentor.	Calculate authoritative score or decide account/capacity eligibility in the browser.
FastAPI / Mentor Discovery module	Authenticate the student; validate filters; orchestrate cache, Neo4j, MySQL, scoring and history writes.	Expose database credentials or let a cached response override server checks.
Neo4j	Generate candidates through Student-Mentor-Skill-Domain-Company-Department paths; return evidence.	Own mentor identity, capacity or final eligibility.
MySQL / InnoDB	Own student and mentor profile data, skill/domain associations, account status, capacity, reviews and recommendation history.	Be bypassed when a graph or cache result is available.
Redis	Cache candidate/recommendation results, stable profile-card data and catalogue values.	Become the authority for whether a mentor can accept a mentee.
Outbox worker	Project committed MySQL profile/catalogue changes to Neo4j; invalidate relevant Redis keys; retry safely.	Project uncommitted changes or lose failed events silently.

3.1 Information ownership

Information	Authority	Use in recommendation
Student identity / account status	MySQL	Authorization and student profile retrieval.
Student skills and domain interests	MySQL; projected to Neo4j	Candidate relationships and score inputs.
Mentor profile, company, skills and expertise	MySQL; projected to Neo4j	Graph discovery, experience and career fit.
Capacity and active allocations	MySQL	Hard eligibility check.
Ratings and session outcomes	MySQL	Mentor quality component.
Candidate paths / relationship evidence	Neo4j projection	Candidate generation and explanation evidence.
Recent recommendation response	Redis cache	Performance only; final eligibility must be rechecked.
Recommendation Run / Recommended Mentor	MySQL	Historical and auditable decision snapshot.

3.2 Architecture principle

MySQL wins when stores disagree

Neo4j can find a mentor who is highly relevant but has already reached capacity. MySQL must exclude that mentor from the final response. Redis may return a previously computed candidate list, but the backend must still perform current eligibility checks before serving it as requestable.

4. Neo4j Recommendation Graph Model

Neo4j is used for relationship-oriented discovery, not as a second authoritative copy of the relational database. Use the same stable MySQL identifier on each graph node so the application can re-check each candidate in MySQL.

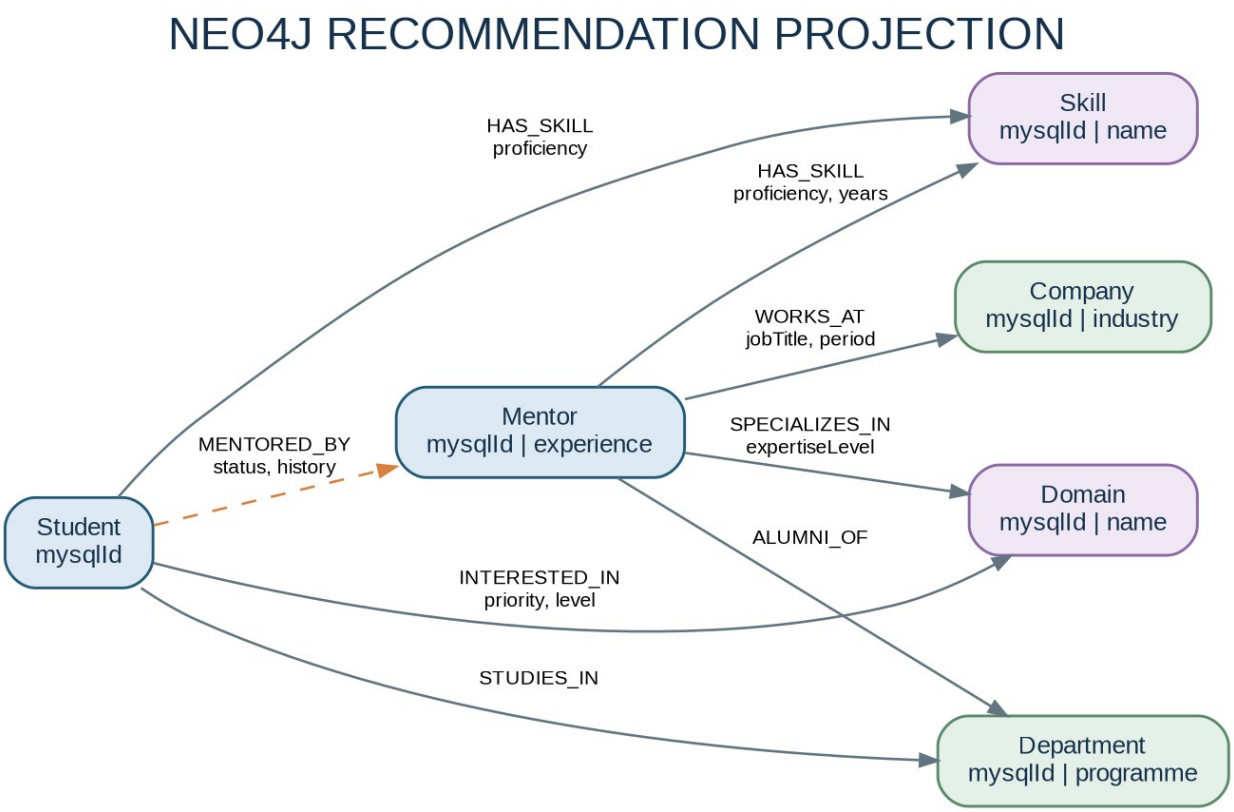


Figure 1. Logical relationship projection used to find and explain mentor candidates.

4.1 Node labels and key properties

Node label	Suggested properties	Purpose
Student	mysqlId, profileVersion, careerGoalTags	Starting node for the signed-in student's discovery.
Mentor	mysqlId, profileVersion, experienceYears, discoverable	Candidate mentor; MySQL remains authoritative.
Skill	mysqlId, name, category	Canonical skill shared by student and mentor.
Domain	mysqlId, name	Mentorship / career domain such as MLOps, Cloud or Data Analytics.
Company	mysqlId, companyName, industry	Professional context for the mentor.
Department	mysqlId, departmentName, programme	Student study and alumni affiliation context.

4.2 Relationships and evidence properties

Relationship	Direction	Useful properties / evidence
HAS_SKILL	Student -> Skill; Mentor -> Skill	proficiency; experienceYears; optional evidenceUpdatedAt.
INTERESTED_IN	Student -> Domain	priority; interestLevel.
SPECIALIZES_IN	Mentor -> Domain	expertiseLevel.
WORKS_AT	Mentor -> Company	jobTitle; startDate; endDate; isCurrent.
STUDIES_IN	Student -> Department	No scoring property required in the initial model.
ALUMNI_OF	Mentor -> Department	No scoring property required in the initial model.
MENTORED_BY	Student -> Mentor	status; startedAt; endedAt; useful for mentoring history.

4.3 Graph modelling rules

- Use mysqlId as the stable cross-database key. Names are display attributes, not identity keys.
- Keep relationship types explicit: HAS_SKILL and SPECIALIZES_IN are not interchangeable.

- Store relationship-specific facts on relationships where they describe the student's or mentor's connection to a skill/domain.
- Use MERGE / idempotent upserts in the projection worker so retries do not create duplicate nodes or relationships.
- Treat graph data as eventually consistent. Keep a source version / update timestamp so stale projections can be detected.

Cypher constraint templates

```
// Illustrative Neo4j uniqueness constraints
CREATE CONSTRAINT student_mysql_id IF NOT EXISTS
FOR (s:Student) REQUIRE s.mysqlId IS UNIQUE;

CREATE CONSTRAINT mentor_mysql_id IF NOT EXISTS
FOR (m:Mentor) REQUIRE m.mysqlId IS UNIQUE;

CREATE CONSTRAINT skill_mysql_id IF NOT EXISTS
FOR (k:Skill) REQUIRE k.mysqlId IS UNIQUE;

CREATE CONSTRAINT domain_mysql_id IF NOT EXISTS
FOR (d:Domain) REQUIRE d.mysqlId IS UNIQUE;
```

The property names above are proposed graph conventions. Align them with the final Neo4j import / projection code.

5. End-to-End Recommendation Pipeline

The sequence below follows the blueprint pipeline: student profile and interests, graph traversal, candidate identification and graph relevance, MySQL eligibility validation, score combination and ranking, persistence, optional cache and display with explanations.

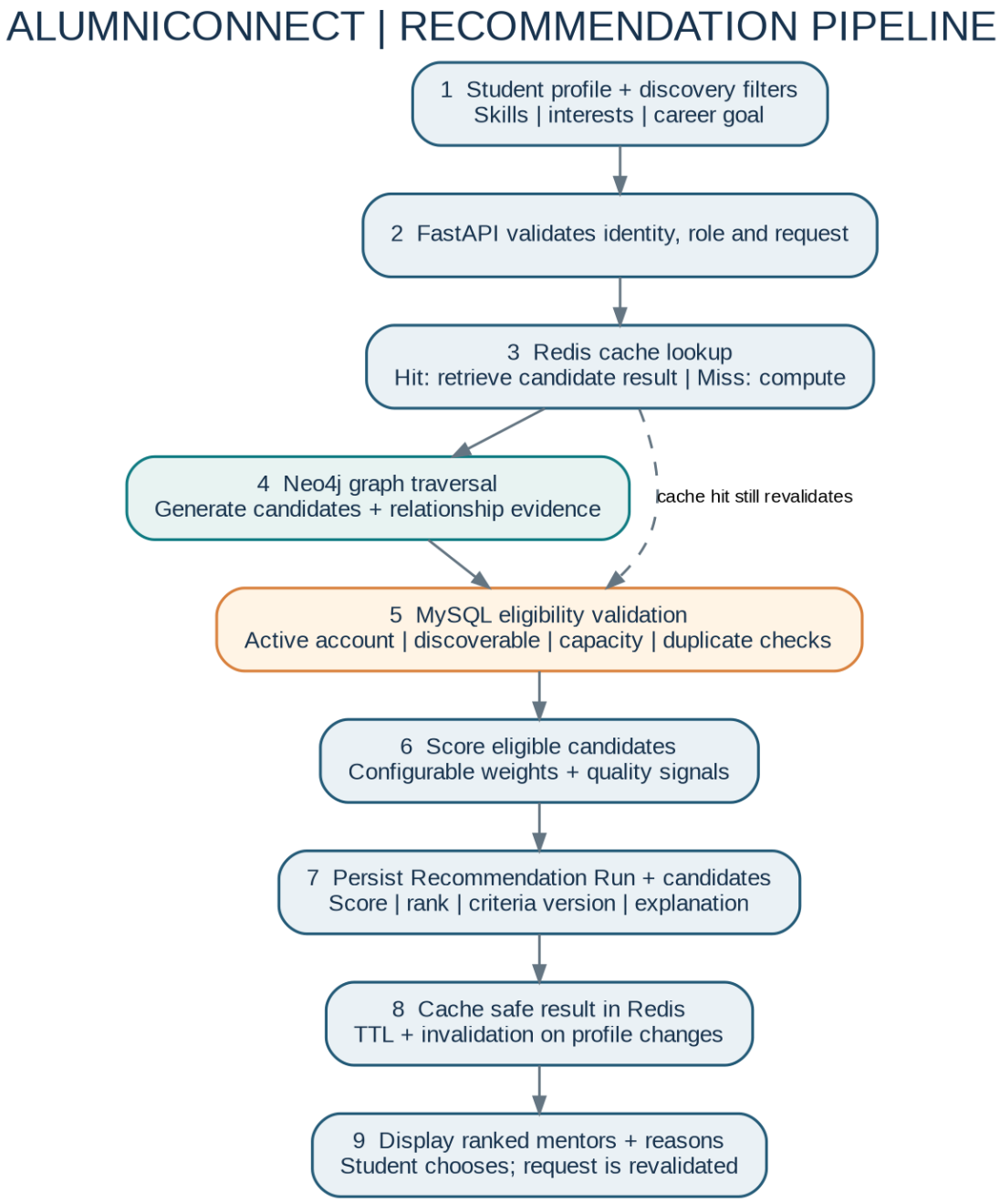


Figure 2. Recommended execution path, including MySQL revalidation even when a cache hit is available.

5.1 Detailed processing steps

Step	Processing	Output
1. Request	Student opens discovery and supplies skill/domain/company/experience filters and optional career goal.	Validated request context and student id.
2. Profile load	FastAPI loads authoritative student skills, domain priorities and career-goal data from MySQL.	Canonical input snapshot.
3. Cache lookup	Check Redis using student, filters and criteria version. A hit may reuse candidate features; a miss starts graph retrieval.	Cached feature payload or cache miss.

Step	Processing	Output
4. Graph retrieval	Neo4j traverses shared skills, interested domains, mentor expertise, company and department paths.	Candidate mentor IDs plus path evidence.
5. Eligibility	MySQL revalidates active status, discoverability, capacity, duplicate-active-allocation rules and selected hard filters.	Eligible candidate set; exclusion reasons for diagnostics.
6. Scoring	Compute dimension scores and weighted total under an explicit criteria version.	Score breakdown and deterministic rank.
7. History write	Insert Recommendation Run and Recommended Mentor rows in MySQL.	Immutable snapshot of run, scores and explanations.
8. Cache write	Cache safe result payload with TTL; source changes invalidate affected keys.	Reusable display result.
9. Display	Return permitted mentor-card fields, rank, overall score, reasons and fit details.	Student can compare and select a mentor.

5.2 Required output contract

Field	Meaning
run_id	Identifier of this recommendation execution.
criteria_version	Exact score configuration used for this run.
generated_at	Timestamp at which the run was calculated.
mentor_id / rank	Candidate identity and rank within the run.
total_score	0-100 weighted match index; not a probability of success.
dimension_scores	Score per matching dimension and its weighted contribution.
reason_codes / explanation	Structured matched facts and a concise human-readable explanation.
eligibility_status	Current MySQL eligibility result; only eligible candidates appear as requestable cards.
selected_flag	Whether the student later selected this candidate from the run.

6. Filtering and Candidate Eligibility

Filtering must be separated from scoring. Filters decide whether a candidate can be shown or whether the student explicitly asked to restrict the search. Scoring decides how strongly each remaining candidate matches.

6.1 Three classes of filter

Class	What it does	Examples
A. Mandatory eligibility gate	Excludes mentors who cannot currently be recommended / cannot accept a mentee.	Inactive account; hidden profile; no remaining capacity; duplicate active student-mentor allocation; self-match.
B. Student-selected hard filter	Restricts results because the student explicitly selected a condition.	Specific domain; selected skill; company / industry; minimum experience; preferred availability window if supplied.
C. Soft ranking signal	Does not automatically exclude a candidate; contributes to score or explanation.	Partial skill overlap; related domain; adjacent career path; relevant industry experience; historical mentor quality.

Do not confuse capacity with appointment availability

Capacity means whether the mentor can accept another mentee / allocation. A published time slot is appointment availability. Do not require an open slot for general mentor discovery unless the student explicitly asks for a date or booking window. Actual booking must separately revalidate the slot in MySQL.

6.2 Mandatory MySQL eligibility checks

Check	Rule	Status
Account status	Mentor account must be active.	[R] operational gate; consistent with current eligibility.
Mentor profile	Mentor profile exists and is discoverable / directory-visible.	[PRD FR-08] public discovery should not expose private profiles.
Role / actor	Candidate has a valid mentor profile and is not the requesting student.	[R] data-integrity gate.
Capacity	Current active allocations are below configured mentor capacity.	[PRD FR-07 / FR-09] check against MySQL.
Duplicate relationship	Exclude or clearly mark a mentor already actively allocated to this student.	[R] prevents duplicate active relationship.
Hard filters	Candidate meets every explicit must-have filter selected by the student.	[R] filter semantics must be defined consistently.
Projection sanity	Candidate id exists in MySQL and relevant source profile is still valid.	[R] protects against stale / orphaned graph nodes.

6.3 User-selected filters

Filter control	Recommended semantics
Mentorship domain	When one or more domains are selected, treat them as strict must-match unless the UI clearly labels them as “preferred”.
Skills	Offer “any selected skill” and “all selected skills” modes. Default to ANY for discovery so the candidate pool is not unnecessarily empty.
Company / industry	Exact company or industry filter only when explicitly selected; do not rank by company prestige.
Minimum experience	A strict floor when the student enters a minimum; otherwise relevant experience remains a score component.
Department / alumni affiliation	Optional filter; also available as contextual evidence. Do not make same-department status mandatory by default.
Availability window	Only apply as a hard filter when the student explicitly supplies a time range. Query current MySQL availability, not cached display state.

6.4 Filter combination rules

- Apply mandatory eligibility checks with AND: every mandatory condition must pass.
- Apply separate filter groups with AND: a selected domain filter AND a selected minimum-experience filter must both pass.
- Within a multi-select skill filter, use the UI-selected ANY or ALL option; show the mode to the student.
- Apply soft signals after filters and use them to rank, not silently remove candidates.
- Persist the exact filter selection and filter semantics in the recommendation run snapshot.

7. Scoring Framework and Configurable Weights

A transparent 0-100 match index is recommended for the first release. The values below are an initial proposed configuration. PRD FR-09 requires that relative weights be configurable; therefore weights must be stored / versioned and not permanently hard-coded.

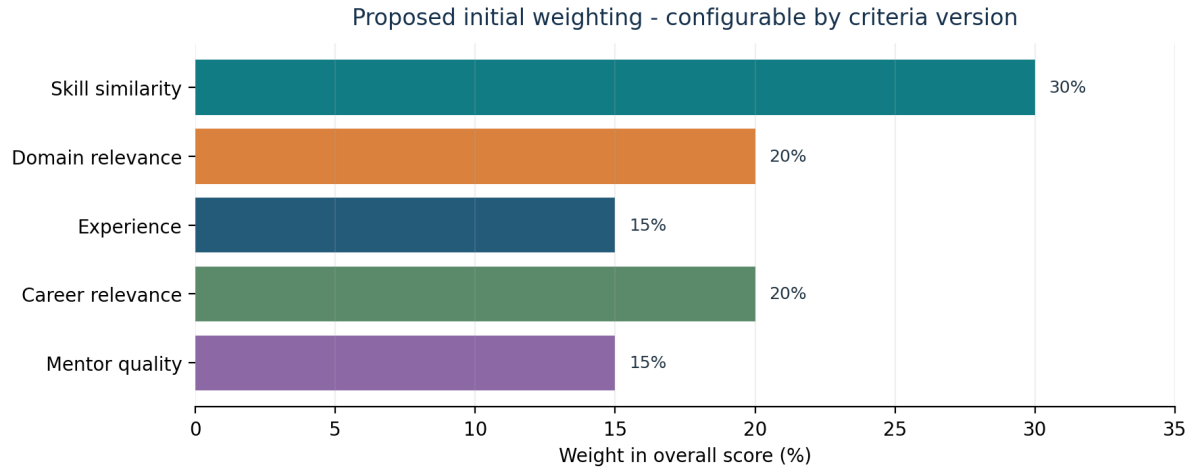


Figure 3. Proposed starting weight allocation. Team may adjust weights after test runs.

Dimension	Default weight	What it measures	Evidence source
Skill similarity	30%	Coverage of student skills by mentor expertise.	Student Skill + Mentor Skill; Neo4j HAS_SKILL paths.
Domain relevance	20%	How closely mentor specialties align with student domain interests and priorities.	Student Domain Interest + Mentor Domain Expertise; INTERESTED_IN / SPECIALIZES_IN.
Relevant experience	15%	Depth of experience that is relevant to matched skills / domains, not just total years.	Mentor Skill experience_years and Mentor Profile experience_years.
Career relevance	20%	Fit between student goal and mentor role, professional domain, company industry and related pathway.	Career-goal taxonomy, job title, industry, domain / alumni links.
Mentor quality	15%	Historical ratings and mentoring outcomes, adjusted for limited review counts.	Mentor Review, Session Feedback and completed-session outcomes.
Capacity / eligibility	Gate	Current ability to accept another mentee; pass/fail before final ranking.	MySQL account status, capacity and active allocation records.

Why capacity is a gate, not a reward score

The PRD names capacity as a recommendation dimension and separately requires candidates failing current MySQL capacity / eligibility checks to be excluded from final results. This design treats capacity as a binary eligibility gate. It avoids showing a full mentor as a low-scoring option that the student cannot actually request.

7.1 Overall score formula

Weighted score formula

```
OverallScore = (wS * SkillScore
        + wD * DomainScore
        + wE * ExperienceScore
        + wC * CareerScore
        + wQ * QualityScore) / (wS + wD + wE + wC + wQ)

Default: 0.30*S + 0.20*D + 0.15*E + 0.20*C + 0.15*Q
```

Each dimension score is normalized to the range 0-100. Since the default weights sum to 100, the result is also between 0 and 100. If a dimension is genuinely unavailable, the scoring service may renormalize over the available dimensions, but the run must record which dimensions were missing and the effective denominator. Do not quietly treat missing data as a zero score.

7.2 Configuration versioning

Configuration field	Purpose
criteria_version	Stable identifier such as mentor-match-v1.
weights	Dimension weights that sum to 1.0 or 100.
score_parameters	Skill proficiency scale, experience target years, career mapping version and quality prior.
effective_from / active_flag	When the criteria version is available for new runs.
created_by / created_at	Administrative audit metadata.
snapshot stored per run	Copy the actual weights and relevant parameters into Recommendation Run so historic scores remain interpretable.

8. Dimension Score Calculation

8.1 Skill similarity - 30%

The blueprint asks how many relevant student skills overlap with mentor expertise. The conceptual model already provides Student Skill and Mentor Skill relationships with proficiency and experience metadata.

Proposed skill coverage formula

```
Let K_s = skills recorded for the student.
For each student skill k, let r_k be its relevance weight (default 1).
Let e_mk be the mentor expertise strength for that skill, normalized 0..1.

SkillScore(m) = 100 * SUM(r_k * e_mk for k in K_s matched by m)
                / SUM(r_k for k in K_s)
```

A possible proficiency normalization is `mentor_proficiency / maximum_proficiency`, capped to 1. Experience in a matched skill may be used to calibrate expertise strength if the team records a consistent scale. Avoid double-counting experience in the skill score and the separate experience dimension.

Example: if a student lists Python, Machine Learning and SQL with equal relevance, and a mentor has strong expertise in Python and Machine Learning but no SQL link, skill coverage is the two matched skills adjusted by expertise strength. The explanation should name the actual matched skills.

8.2 Domain relevance - 20%

The student's domain-interest association contains priority / interest-level information. Mentor domain expertise contains an expertise level. This is a good place to use the relationship metadata rather than merely checking whether a domain name appears in a free-text biography.

Proposed domain alignment formula

```
Let D_s be the student preferred domains.
Let p_d be the student priority weight for domain d.
Let x_md be mentor expertise for domain d, normalized 0..1.

DomainScore(m) = 100 * SUM(p_d * x_md for matching domains)
                 / SUM(p_d for all student domains)
```

If the student sets Machine Learning as a higher-priority interest than Cloud Computing, a mentor specializing in Machine Learning contributes more strongly. Keep the scale for priority and expertise versioned so scores are comparable across runs.

8.3 Relevant experience - 15%

Use experience that is related to the matched skills / domains where possible. The Mentor Profile's total experience years can be a fallback, but should not automatically be treated as relevant to every student goal.

Proposed experience normalization

```
RelevantYears(m) = weighted years of experience in matched
                   skills / domains (fallback: profile experience).

ExperienceScore(m) = MIN(100,
                         100 * RelevantYears(m) / TargetYears)
```

A starting `TargetYears` of 5 is used in the worked example only. The team should choose a configurable cap for its student population. A cap prevents very high experience values from dominating the score indefinitely.

8.4 Career relevance - 20%

This dimension asks whether the mentor's professional background fits the student's career goal. The current conceptual model includes a student career goal and mentor job title, company and industry. For v1, map these to a small, controlled career-domain taxonomy.

Evidence relationship	Illustrative rule score
Exact career domain / job-family alignment	100
Strong adjacent pathway or same target industry with related role	75
Broadly related professional path	50
No clear evidence of relevance	0

These values are suggested rule bands, not measured probabilities. Store the mapping version and the evidence used. Do not infer relevance from employer prestige alone. A later semantic-search extension may add a semantic similarity signal, but the v1 rule-based score must remain understandable.

8.5 Mentor quality - 15%

Quality should use both historical ratings and outcomes while avoiding the mistake of treating one five-star review as definitive. A Bayesian-smoothed rating is a reasonable proposed approach for mentors with few reviews.

Proposed mentor-quality formula

```
AdjustedRating = (v / (v + m)) * MentorAverage
               + (m / (v + m)) * PlatformAverage

RatingScore = 100 * (AdjustedRating - 1) / 4   // 1..5 scale

QualityScore = 0.80 * RatingScore
             + 0.20 * CompletedOutcomeRate
```

Here `v` is the number of valid reviews and `m` is a configurable prior review count (example default: 5). `PlatformAverage` is the current platform mean rating. `CompletedOutcomeRate` may be defined as completed sessions divided by completed plus no-show sessions; participant-cancelled sessions should not be treated as a mentor failure by default. If outcome data is not reliable or unavailable, omit that part and renormalize the quality subscore. New mentors should receive a neutral prior and a "limited history" disclosure, not an automatic zero.

Do not call the score a success probability

An overall score of 91 means that the candidate has a strong relative fit under the selected criteria version. It does not mean the student has a 91% chance of a successful mentorship.

9. Worked Recommendation Example

The blueprint uses an illustrative student interested in Machine Learning, MLOps and Cloud Computing, with current skills in Python, Machine Learning and SQL. It also shows three sample mentors: an ML Engineer with 5 years, a Cloud Engineer with 4 years and a Data Scientist with 3 years. The numerical sub-scores below are synthetic examples added only to demonstrate the proposed calculation.

9.1 Candidate evidence

Candidate	Profile evidence from the illustrative scenario	Likely reason pattern
Mentor A	ML Engineer; 5 years; Python, ML, MLOps, Docker; Machine Learning domain.	Strong shared skills, strong preferred-domain match, strong career alignment.
Mentor B	Cloud Engineer; 4 years; AWS, Docker, Kubernetes; Cloud Computing domain.	Strong domain relevance; less direct overlap with current student skills.
Mentor C	Data Scientist; 3 years; Python, SQL, Data Science; Data Analytics domain.	Some shared skills; weaker direct preferred-domain alignment.

9.2 Example component scores and weighted contributions

Dimension	Weight	Mentor A score / contribution	Mentor B score / contribution	Mentor C score / contribution
Skill similarity	30%	90 / 27.00	55 / 16.50	70 / 21.00
Domain relevance	20%	100 / 20.00	90 / 18.00	45 / 9.00
Experience	15%	90 / 13.50	80 / 12.00	60 / 9.00
Career relevance	20%	95 / 19.00	75 / 15.00	60 / 12.00
Mentor quality	15%	80 / 12.00	88 / 13.20	90 / 13.50
TOTAL	100%	91.50	74.70	64.50

Mentor A ranks first in this illustrative run because it provides a strong fit across skill, domain, experience and career relevance. Mentor B remains a strong alternative for a student prioritizing Cloud Computing. Mentor C has a good skill overlap and quality signal, but a weaker domain / career alignment in this particular profile.

9.3 Interpretation rules

- Rank only candidates who pass current MySQL eligibility checks.
- Display component scores so that the student can understand why a candidate ranked first.
- Do not hide the second-best candidate simply because one mentor has a higher score.
- Show data limitations such as few reviews or missing experience details.
- When weights or profile inputs change, create a new recommendation run rather than rewriting the old score.

10. Recommendation Explanation Design

A score without evidence is not enough. The blueprint explicitly recommends displaying reasons such as shared skills, preferred-domain specialization, relevant industry experience and mentoring students in similar career areas.

10.1 Explanation components

Explanation element	What the student sees	Generated from
Summary	“Strong match for your Machine Learning and MLOps interests.”	Highest-scoring dimensions and matching tags.
Shared skills	“Shares Python and Machine Learning expertise with your profile.”	Neo4j HAS_SKILL paths and MySQL skill rows.
Domain match	“Specializes in Machine Learning, one of your priority domains.”	INTERESTED_IN + SPECIALIZES_IN relationship evidence.
Experience	“Has 5 years of relevant ML engineering experience.”	Mentor Skill / Mentor Profile experience records.
Career context	“Current role aligns with your MLOps career goal.”	Job-family / industry / domain mapping.
Mentoring history	“Has prior mentoring activity in related career areas.”	MENTORED_BY and session / outcome history when available.

Explanation element	What the student sees	Generated from
Quality signal	“Strong historical feedback across 8 completed sessions.”	MySQL review and session aggregates; disclose sample size.

10.2 Sample student-facing card

Mentor A - ML Engineer | Match score: 91.5 / 100

Recommended because this mentor shares Python and Machine Learning expertise with your profile, specializes in your high-priority Machine Learning domain, and has five years of relevant ML engineering experience. Their professional role aligns closely with your MLOps direction. Historical feedback is positive; review and session counts are shown so you can judge the evidence.

10.3 Explanation payload design

Example explanation JSON (illustrative fields)

```
{
  "summary": "Strong match for Machine Learning and MLOps interests",
  "reason_codes": [
    "SKILL_OVERLAP", "DOMAIN_MATCH",
    "RELEVANT_EXPERIENCE", "CAREER_ALIGNMENT",
    "MENTOR_QUALITY"
  ],
  "matched_skills": ["Python", "Machine Learning"],
  "matched_domains": ["Machine Learning"],
  "relevant_experience_years": 5,
  "career_evidence": "ML Engineer role aligns with MLOps goal",
  "quality_evidence": {"rating": 4.6, "review_count": 8},
  "score_breakdown": {
    "skill": {"score": 90, "weight": 0.30, "contribution": 27.0},
    "domain": {"score": 100, "weight": 0.20, "contribution": 20.0}
  }
}
```

10.4 Explanation generation rules

- Build explanations from stored evidence / reason codes, not from a free-form model that can invent qualifications.
- Show at most 3-4 strongest reasons in the initial card; allow the student to expand the full score breakdown.
- Display rating counts and the limited-history label where sample size is small.
- Do not reveal private contact details, private account fields or another student’s mentorship history.
- If the profile or graph evidence is stale, do not state that a match is current until authoritative data is refreshed.

11. Recommendation History and Relational Data Design

The Conceptual ER Diagram introduces two separate discovery-history entities: Recommendation Run and Recommended Mentor. A run stores one execution; each Recommended Mentor row records one candidate returned by that run, including rank, score, explanation and selected flag.

Entity	Proposed key attributes	Purpose
Recommendation Run	run_id; student_id; generated_at; criteria_version; input_snapshot	One execution for one student, with immutable inputs and scoring configuration.
Recommended Mentor	recommendation_id; run_id; mentor_id; rank; score; explanation; selected_flag	One candidate from a particular run, including why and how it ranked.
Criteria configuration [R]	criteria_version; weights; parameters; effective_from	Manage adjustable weights and calculation parameters outside source code.
Selection linkage [R]	selected_at; request_id (optional FK)	Connect a selected recommendation to the request that the student submits.

11.1 Recommended relationships

Relationship	Cardinality	Interpretation
Student -> Recommendation Run	1 : 0..*	A student may run discovery repeatedly.
Recommendation Run -> Recommended Mentor	1 : 0..*	One run may return zero or many candidates.
Mentor -> Recommended Mentor	1 : 0..*	A mentor may appear in many different runs.
Recommended Mentor -> Mentorship Request	0..1 (proposed link)	A candidate may be selected and produce a request; the request remains its own business record.

11.2 Why the history must be immutable

- A student may update skills or domain priorities and receive a different list a week later.
- The team must be able to reconstruct which candidates, scores, filters and weights were used at the time.
- A later criteria change should create a new criteria_version, not silently change old ranking values.
- Selection and later outcome fields are appended / updated as outcome events while the original score and explanation are preserved.

11.3 Input snapshot fields

Snapshot group	Store enough to reproduce the calculation
Student matching input	Student id, relevant skill ids, normalized domain priorities, career-goal tags and profile version.
User-selected filters	Selected domain / skill / company / industry / min-experience / date-window filters and ANY / ALL skill semantics.
Scoring configuration	Criteria version, weights, normalization parameters, quality prior and career taxonomy version.
Graph freshness	Optional graph projection watermark / event version used by the run.
Privacy	Store only fields necessary for reproducibility; do not copy contact details or full private biographies into the snapshot.

12. MySQL Storage Templates

The SQL below is an implementation template, not a claim that the final physical schema already uses these exact table or column names. The foreign-key target names must be aligned with the approved relational schema and normalization work.

12.1 Recommendation Run - illustrative DDL

```
CREATE TABLE recommendation_run (
  run_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  generated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  criteria_version VARCHAR(32) NOT NULL,
  input_snapshot JSON NOT NULL,
  filter_snapshot JSON NULL,
  weight_snapshot JSON NOT NULL,
  graph_watermark VARCHAR(100) NULL,
  candidate_count INT NOT NULL DEFAULT 0,
  result_count INT NOT NULL DEFAULT 0,
  CONSTRAINT fk_run_student
    FOREIGN KEY (student_id) REFERENCES student_profile(student_id),
  INDEX idx_run_student_time (student_id, generated_at)
) ENGINE=InnoDB;
```

12.2 Recommended Mentor - illustrative DDL

```
CREATE TABLE recommended_mentor (
   recommendation_id BIGINT PRIMARY KEY AUTO_INCREMENT,
   run_id BIGINT NOT NULL,
   mentor_id BIGINT NOT NULL,
   rank_no INT NOT NULL,
   total_score DECIMAL(5,2) NOT NULL,
   skill_score DECIMAL(5,2) NOT NULL,
   domain_score DECIMAL(5,2) NOT NULL,
   experience_score DECIMAL(5,2) NOT NULL,
   career_score DECIMAL(5,2) NOT NULL,
   quality_score DECIMAL(5,2) NOT NULL,
   explanation_json JSON NOT NULL,
   reason_codes JSON NOT NULL,
   selected_flag BOOLEAN NOT NULL DEFAULT FALSE,
   selected_at DATETIME(6) NULL,
   request_id BIGINT NULL,
   CONSTRAINT fk_rec_run FOREIGN KEY (run_id)
     REFERENCES recommendation_run(run_id),
   CONSTRAINT fk_rec_mentor FOREIGN KEY (mentor_id)
     REFERENCES mentor_profile(mentor_id),
   CONSTRAINT fk_rec_request FOREIGN KEY (request_id)
     REFERENCES mentorship_request(request_id),
   CONSTRAINT uq_run_mentor UNIQUE (run_id, mentor_id),
   CONSTRAINT uq_run_rank UNIQUE (run_id, rank_no),
   CONSTRAINT uq_rec_request UNIQUE (request_id),
   CONSTRAINT ck_total_score CHECK (total_score BETWEEN 0 AND 100),
   INDEX idx_rec_mentor (mentor_id, run_id)
) ENGINE=InnoDB;
```

12.3 Storage and integrity guidance

- Use DECIMAL rather than FLOAT for stored display scores so the saved score and displayed score remain stable.
- Keep run_id + mentor_id unique so a duplicate graph path cannot create duplicate candidate rows for the same run.
- Keep rank unique within a run if every final result must have one distinct rank.
- Store explanation_json and reason_codes so UI wording can improve later without losing the original evidence.
- Use JSON for snapshots that vary by criteria version; retain core queryable attributes in typed columns.
- Do not overwrite old runs when the student changes their profile; create a new run.

If the team chooses to persist excluded graph candidates for debugging, use a separate candidate-evaluation table with eligibility status and exclusion reason. Do not mix excluded candidates with the final Recommended Mentor list unless the API clearly distinguishes them.

13. Neo4j Candidate Retrieval - Cypher Examples

Use Neo4j to return mentor ids and the graph evidence that caused each candidate to be discovered. The backend can merge results from separate skill, domain and professional-context traversals by mentor id; this keeps query logic readable and makes explanation evidence explicit.

13.1 Skill-based candidate discovery

Cypher A - shared skills

```
MATCH (s:Student {mysqlId: $studentId})
    -[:HAS_SKILL]->(sk:Skill)
    <-[hs:HAS_SKILL]-(m:Mentor)
RETURN
  m.mysqlId AS mentorId,
  collect(DISTINCT {
    skillId: sk.mysqlId,
    skillName: sk.name,
    proficiency: hs.proficiency,
    experienceYears: hs.experienceYears
  }) AS matchedSkills,
  count(DISTINCT sk.mysqlId) AS sharedSkillCount
ORDER BY sharedSkillCount DESC;
```

13.2 Domain-based candidate discovery

Cypher B - shared interest / expertise domains

```
MATCH (s:Student {mysqlId: $studentId})
    -[interest:INTERESTED_IN]->(d:Domain)
    <-[expertise:SPECIALIZES_IN]-(m:Mentor)
RETURN
  m.mysqlId AS mentorId,
  collect(DISTINCT {
    domainId: d.mysqlId,
    domainName: d.name,
    studentPriority: interest.priority,
    mentorExpertise: expertise.expertiseLevel
  }) AS matchedDomains
ORDER BY size(matchedDomains) DESC;
```

13.3 Professional-context discovery

Cypher C - company / industry evidence (only when relevant filters are used)

```
MATCH (m:Mentor)-[job:WORKS_AT]->(c:Company)
WHERE c.mysqlId IN $selectedCompanyIds
  OR c.industry IN $selectedIndustries
RETURN m.mysqlId AS mentorId,
    collect(DISTINCT {
      companyId: c.mysqlId,
      companyName: c.companyName,
      industry: c.industry,
      jobTitle: job.jobTitle,
      isCurrent: job.isCurrent
    }) AS careerEvidence;
```

13.4 Merge and de-duplicate candidates

- The FastAPI discovery module collects mentorId values from the skill, domain and optional career traversals.
- Merge by mysqlId and de-duplicate evidence lists by skillId / domainId / companyId.
- Retain the graph evidence used to build score components and explanation text.
- Pass only candidate ids and required feature evidence to the MySQL validation stage; do not treat graph properties as authoritative capacity or ratings.

Avoid a name-based join

Use the stable mysqlId property to map Neo4j candidates back to MySQL. Names, company strings and display labels can change or collide; they should not be used as relational join keys.

14. MySQL Eligibility Validation and Final Ranking

After graph retrieval, validate current eligibility against MySQL. The PRD explicitly requires the final candidate list to exclude candidates who fail current MySQL eligibility or capacity checks.

14.1 SQL-style eligibility template

Illustrative eligibility query - adapt names and status model to final schema

```
SELECT m.mentor_id, m.experience_years, m.capacity,
       u.account_status
FROM mentor_profile AS m
JOIN user_account AS u ON u.user_id = m.user_id
WHERE m.mentor_id IN (/* Neo4j candidate ids */)
  AND u.account_status = 'ACTIVE'
  AND m.is_discoverable = 1
  AND (
      SELECT COUNT(*)
      FROM mentor_allocation AS a
      WHERE a.mentor_id = m.mentor_id
        AND a.status = 'ACTIVE'
    ) < m.capacity
  AND NOT EXISTS (
      SELECT 1
      FROM mentor_allocation AS a2
      WHERE a2.mentor_id = m.mentor_id
        AND a2.student_id = :student_id
        AND a2.status = 'ACTIVE'
    );
```

This query shows the logical checks only. If availability or company filters are selected, apply their authoritative predicates as additional conditions. Index foreign keys / status columns used by these predicates after validating with EXPLAIN.

14.2 Score combination and ranking

Illustrative ranking expression for the v1 configuration

```
SELECT mentor_id,
       ROUND(
         0.30 * skill_score
       + 0.20 * domain_score
       + 0.15 * experience_score
       + 0.20 * career_score
       + 0.15 * quality_score, 2
       ) AS total_score
FROM eligible_candidate_scores
ORDER BY total_score DESC,
         domain_score DESC,
         skill_score DESC,
         mentor_id ASC;
```

Use a stable final tie-breaker (mentor_id) so repeated runs with identical inputs do not produce random order changes. The exact tie-break policy is a recommended implementation choice.

14.3 Index / query optimisation candidates

Query access pattern	Potential index / constraint
Student -> Student Skill	Composite primary / unique key (student_id, skill_id); index skill_id for reverse traversal in SQL.
Mentor -> Mentor Skill	Composite primary / unique key (mentor_id, skill_id); index skill_id.
Student Domain Interest	Composite unique key (student_id, domain_id); index domain_id.
Mentor Domain Expertise	Composite unique key (mentor_id, domain_id); index domain_id.
Active mentor allocations by mentor	Index (mentor_id, status); consider student_id in a composite index for duplicate checks.
Recommendation history	Index (student_id, generated_at) on run; index (run_id, rank_no) on candidate rows.
Mentor reviews	Index (mentor_id, created_at) or (mentor_id) for aggregates, depending on reports.

Demonstrate query optimization by comparing the discovery / eligibility query plan before and after indexes with EXPLAIN. Avoid adding every possible index: measure using the project dataset and retain indexes that help the actual access path.

15. Redis Recommendation Cache and Performance

Redis is a performance layer. The Application Architecture proposes caching recent recommendation results and popular mentor ranking views, using cache-aside, TTL and invalidation. The cache must remain rebuildable.

15.1 Suggested key design

Illustrative cache key / value

```
recommendation:v1:{student_id}:{filter_hash}:{criteria_version}

Example value:
{
  "run_id": 812,
  "mentor_ids": [104, 209, 311],
  "score_version": "mentor-match-v1",
  "generated_at": "2026-11-02T18:00:00Z",
  "profile_version": 17
}
```

Cache policy	Recommended behaviour
Key partitioning	Include student id, normalized filter hash and criteria version. Include profile / graph version where available.
TTL	Start with a short configurable TTL (example: 5-10 minutes); tune under test. This is a proposed value, not a source requirement.
Cache hit	Use cached candidate / feature data, but revalidate current MySQL eligibility before returning requestable cards.
Cache miss	Run graph discovery, SQL checks, scoring and history persistence; then cache a safe response payload.
Invalidation	Invalidate when student skills/interests/career goal, mentor profile/skills/domains, mentor capacity, ratings or catalogue mappings change.
Privacy	Do not cache private fields in a shared / publicly accessible keyspace; include authorization in the retrieval path.

15.2 Invalidation triggers

Committed MySQL event	Required secondary action
StudentProfileUpdated / StudentSkillsUpdated	Invalidate this student's recommendation keys.
MentorProfileUpdated / MentorSkillsUpdated	Invalidate mentor-card keys and recommendation sets that may contain the mentor.
MentorDomainExpertiseUpdated	Invalidate graph projection and relevant recommendation keys.
MentorCapacityChanged / AllocationCreated / AllocationEnded	Revalidate candidate eligibility; invalidate relevant cached rankings.
MentorReviewCreated / SessionOutcomeRecorded	Refresh mentor-quality signals and cache entries.
SkillOrDomainCatalogueChanged	Refresh graph/catalogue projections and affected recommendation caches.

Cache is never booking authority

A cached recommendation can become stale after another student creates an allocation or a mentor changes capacity. Current MySQL must determine whether the mentor is still requestable. Booking slot correctness is handled by the separate atomic booking transaction, not by this recommendation cache.

16. Cross-Database Consistency and Event Processing

MySQL is strongly authoritative for business records, Redis is disposable cache / temporary state, and Neo4j is an eventually consistent projection. The application should not independently write to MySQL and Neo4j and assume both writes will succeed.

16.1 Mentor skill update example

Step	Action	Consistency result
1	Mentor changes skills through the Profile / Skill module.	Input is validated against managed Skill records.
2	One MySQL transaction updates Mentor Skill rows and writes a MentorSkillsUpdated outbox event.	Either the data change and event both commit, or neither does.
3	Background worker reads the committed event.	Only committed events are processed.
4	Worker upserts graph nodes / relationships in Neo4j.	Graph projection catches up asynchronously.
5	Worker invalidates affected Redis cache keys.	Next lookup reloads current recommendation data.
6	Worker records processed / failed status.	Retries and projection health are observable.

16.2 Worker reliability requirements

- Use stable event identifiers and idempotent graph upsert / cache invalidation operations.
- Track pending, processed and failed attempts; retain error detail for investigation.
- Retry transient errors with bounded backoff.
- Do not discard a failed graph projection event silently.
- Treat graph lag as recommendation freshness risk, not permission to skip MySQL validation.

16.3 Graph freshness in a recommendation run

Optionally store the latest processed outbox event id / graph watermark in Recommendation Run. This lets an administrator understand whether the recommendation was generated from a fully up-to-date projection or while a sync backlog existed. Where possible, show a freshness notice internally rather than exposing technical details on the student card.

17. FastAPI Module and API Contract

Keep Mentor Discovery as a clearly bounded backend module within the FastAPI modular monolith. Routers should validate request shape and authentication context; the application service coordinates graph retrieval, eligibility, scoring and persistence.

17.1 Module responsibilities

Layer	Recommended responsibility
Router / API boundary	Authenticate caller, validate path and query parameters, enforce student ownership, return response schema.
Discovery application service	Load canonical student profile, normalize filters, invoke Neo4j candidate provider, call eligibility repository, score and rank.
Scoring service	Pure deterministic functions for each dimension and overall score; load criteria version.
Recommendation repository	Persist Recommendation Run and Recommended Mentor; fetch historical runs and selection outcome.
Graph repository	Run parameterized Cypher queries and return graph evidence by mysqlId.
Cache adapter	Read / write / invalidate Redis entries; no authority over final business eligibility.

17.2 Illustrative API endpoints

Method + route	Purpose	Authorization
POST /recommendations/runs	Create a new recommendation run using the authenticated student's profile and selected filters.	Student; student id from auth context.
GET /recommendations/runs/{run_id}	Retrieve a saved run, candidates and score explanations.	Owner student; admin via explicit report permission.
POST /recommendations/runs/{run_id}/select	Mark a candidate selected and initiate mentorship request creation after fresh MySQL checks.	Owner student.
GET /students/me/recommendations/latest	Retrieve latest run; revalidate eligibility before presenting requestable actions.	Authenticated student.
GET /admin/recommendations/metrics	View aggregated runs, selected candidates and subsequent outcomes.	Administrator / designated reporting permission.

17.3 Request payload example

Illustrative POST body - do not trust student_id from client input

```
{
  "filters": {
    "domain_ids": [12, 18],
    "skill_ids": [4, 9, 15],
    "skill_match_mode": "ANY",
    "company_ids": [],
    "industries": [],
    "minimum_experience_years": 2
  },
  "criteria_version": "mentor-match-v1",
  "limit": 10
}
```

17.4 Select-to-request flow

1. Student selects a Recommended Mentor row from a stored run.
2. Backend confirms the run belongs to the authenticated student and the selected mentor appeared in that run.
3. Backend re-checks mentor account status, capacity, existing allocation and any duplicate pending request in MySQL.
4. Backend creates a Pending Mentorship Request and selection linkage in one transaction where practical.
5. The committed transaction writes a transactional outbox event; notification delivery occurs after commit.

18. Edge Cases and Degraded Behaviour

Scenario	Expected behaviour
Student has no skills	Show a profile-completion prompt and continue with available domain / career signals where possible; disclose lower confidence.
Student has no domain interests	Use skill and career signals; prompt the student to add interests; do not invent a preferred domain.
Mentor has no review history	Use a neutral platform prior; label “new / limited feedback history”. Do not give an automatic zero.
Mentor has missing experience details	Mark experience evidence incomplete; avoid treating null as zero years unless product policy explicitly states that.
No graph candidates	Return an empty result with filter-broadening suggestions; do not fabricate candidates.
Graph is unavailable	Use a clearly defined degraded path only if a safe, fresh MySQL/Redis candidate fallback exists; otherwise return a retryable service error.
Neo4j is stale	MySQL removes inactive / ineligible candidates; show only current candidates. Trigger projection repair / reprocessing.
Redis is unavailable	Bypass cache and run the normal discovery pipeline; Redis failure must not change correctness.
Mentor reaches capacity after cache fill	MySQL eligibility check excludes the mentor at read time; invalidate the cache.
Criteria version changes	Use the new version for new runs; preserve old scores / weights for historical runs.
Same overall score	Apply the documented stable tie-break order and retain deterministic ranking.

19. Security, Privacy and Responsible Ranking

19.1 Data access

- Only authenticated students may request recommendations for their own student profile unless an explicitly authorized administrative use case exists.
- The directory response should expose only information permitted for public mentor discovery; do not return email, phone, private notes or account credentials.
- Administrative recommendation analytics should be aggregate-first and permission controlled.
- Parameterized SQL and Cypher should be used; do not concatenate user-supplied filter strings into query text.

19.2 Responsible scoring principles

Principle	Implementation recommendation
Relevant evidence only	Use declared skills, domains, professional experience, career tags and mentoring outcomes; do not introduce protected or unrelated personal attributes.
No prestige-only ranking	Company information should contribute only through the relevance of industry / role / career context, not employer brand status.
Limited history awareness	A new mentor must not be automatically pushed to the bottom for having few reviews; use a neutral prior and show sample size.
Human choice	Recommendations are suggestions; the student chooses whether to submit a mentorship request.
Transparent score	Show important matching dimensions, not just one opaque total.
Auditability	Retain criteria version, input/filter snapshot, dimensions and selection outcome.

19.3 Profile completeness and data confidence

A recommendation score is only as reliable as its profile evidence. The system should record data-quality flags such as missing student skills, missing mentor experience, no review history or stale graph projection. These flags should influence explanation language and internal evaluation; they should not silently alter score meaning.

20. Evaluation Metrics and Acceptance Tests

Recommendation evaluation should measure both technical correctness and useful outcomes. A high score alone does not demonstrate that the right mentor was found; compare recommendations with student selections, mentor acceptance and actual session outcomes.

20.1 Product / recommendation metrics

Metric	Definition / interpretation
Candidate coverage	How many eligible mentors can be surfaced for a student / domain.
Precision@K	Among the top K mentors, how many are considered relevant by student feedback or later interaction evidence.
Selection rate	Selected mentor candidates divided by displayed eligible recommendations.
Request conversion	Mentorship requests submitted from recommended candidates divided by recommendation views.
Acceptance rate	Accepted requests divided by submitted requests from recommendations.
Session completion rate	Completed sessions divided by sessions that reached a scheduled / outcome state, using documented treatment of cancellations.
Post-session quality	Mentor reviews and structured feedback after completed sessions; report sample counts alongside averages.
Freshness / correctness	Rate of stale or ineligible mentors that appear in final results; target should be zero for eligibility violations.

20.2 Acceptance test catalogue

ID	Test	Expected result
REC-01	Student with shared skills discovers a mentor via HAS_SKILL.	Candidate appears with matched skill evidence.
REC-02	Student interested in a domain discovers a mentor specializing in it.	Candidate appears with domain reason and score.
REC-03	Mentor has full allocation capacity in MySQL.	Mentor is excluded from final eligible results.
REC-04	Mentor account is inactive / profile hidden.	Mentor is excluded.
REC-05	Student selects minimum experience.	Mentors below the selected floor are filtered out.
REC-06	Weights are changed under a new criteria version.	New run uses new weights; old run values remain unchanged.
REC-07	Same inputs and same criteria version are run twice.	Scores and deterministic ordering are reproducible.
REC-08	Student updates skills in MySQL.	Outbox eventually updates Neo4j and invalidates recommendation cache.
REC-09	A cache hit exists but mentor capacity has changed.	MySQL revalidation removes the mentor from requestable final results.
REC-10	Mentor has no reviews.	Neutral prior / limited-history disclosure; not a zero rating.
REC-11	Recommendation is selected.	selected_flag and selection / request linkage are saved.
REC-12	Student asks for explanation.	Reason fields match actual graph / SQL evidence; no invented claims.
REC-13	Student requests another student’s run.	Authorization denies access.
REC-14	No eligible candidates exist.	Empty result with filter-broadening guidance; no fabricated card.

20.3 Performance target from the PRD

Proposed PRD performance baseline

The supplied PRD proposes recommendation latency within 4 seconds at p95 for an initial test profile of up to 50 concurrent users. Treat this as a project target to load-test and report with dataset size, environment, p50 / p95 latency, throughput and error rate - not as a guaranteed result.

21. Implementation Roadmap and DBMS Syllabus Mapping

21.1 Implementation stages

Stage	Build	Exit condition
1. Profile and graph projection	Student / mentor skills, domain interests / expertise, company / department links; outbox projection to Neo4j.	A sample student can traverse skill and domain paths to candidate mentor IDs.
2. Eligibility filters	MySQL status, discoverability, capacity and duplicate-allocation checks; explicit selected filters.	Ineligible candidates are consistently excluded, including when Redis contains old results.
3. Deterministic scoring	Implement five dimension scores and the configurable criteria version.	Unit tests reproduce known score examples and ranking order.
4. Explanation and history	Recommendation Run, Recommended Mentor rows, explanation JSON, selected flag and run retrieval API.	A run can be reconstructed and compared after a profile / weight change.
5. Redis caching	Cache-aside, TTL, invalidation and cache-hit revalidation.	Cache hit/miss/expiry and profile-change invalidation are demonstrated.
6. Quality evaluation	Generate sample data; calculate Precision@K / request conversion / acceptance / completion.	Team can compare versions and explain what improved or regressed.
7. Optional Phase 2	Redis Stack vector search to support semantic career-goal matching.	Graph + vector candidates are merged, then MySQL performs the same final eligibility checks.

21.2 Mapping to the DBMS course

Unit	Course concept	AlumniConnect demonstration
Unit 1	ER model, entity relationships, keys, SQL and constraints.	Student Skill / Mentor Skill / Domain junctions; Recommendation Run and Recommended Mentor relationships.
Unit 2	Joins, aggregates, views, triggers, procedures, CTEs, windows, full-text search.	Mentor rating aggregates, ranked discovery reports, recommendation history and run analytics.
Unit 3	Functional dependencies, normal forms, query optimization.	Normalized run/candidate history; indexes and EXPLAIN for eligibility / report queries.
Unit 4	Transactions, concurrency, locking, key-value and graph databases.	Transactional outbox; Neo4j candidate traversal; Redis recommendation cache and invalidation; optional vector search.

21.3 Recommended viva explanation

How to explain the recommendation engine

“We use Neo4j to discover mentor candidates through shared skills, domains and professional connections. We then validate each candidate in MySQL because MySQL owns current account status and mentorship capacity. Eligible candidates receive a transparent weighted score across skill similarity, domain relevance, experience, career relevance and mentor quality. We save each run and its explanations in MySQL, and use Redis only to cache recent results. Selecting a recommendation creates a mentorship request; it does not create an allocation until the mentor accepts.”

22. Team Decisions to Finalise Before Coding

The following policies are not numerically fixed in the supplied source documents. The team should decide and store them as configuration rather than embedding them in multiple code paths.

Decision	Recommended starting point	Owner / status
Initial weight set	30 skill / 20 domain / 15 experience / 20 career / 15 quality.	Team approval required.
Student skill relevance	Equal importance by default; optionally let students mark priority skills.	Product / team decision.
Experience target cap	Start with 5 relevant years for the worked example; test and tune.	Team decision.
Career taxonomy	Managed categories for job family, industry and domain relationships.	Admin / team decision.

Decision	Recommended starting point	Owner / status
Quality prior m	Illustrative prior count 5 and platform mean rating; review after data volume grows.	Team decision.
Minimum evidence	Define when quality metrics are reliable enough to show; label low sample sizes.	Team decision.
Capacity semantics	Count active allocations against mentor capacity; confirm any paused / ended state semantics.	Business-rule decision.
Recommendation list size	Select a default number of top candidates, e.g. 10, and allow pagination.	UX decision.
Cache TTL	Start with 5-10 minutes and verify freshness / performance under test.	Engineering decision.
History retention	Define retention / deletion policy while preserving required auditability.	Admin / privacy decision.

23. Glossary

Term	Meaning
Candidate generation	Finding a pool of mentor ids through graph paths.
Eligibility gate	A mandatory test that removes candidates who are not currently requestable.
Dimension score	A normalized 0-100 score for one matching factor.
Weighted contribution	Dimension score multiplied by its configured weight.
Criteria version	A named and immutable set of weights and score parameters.
Recommendation Run	One recorded calculation for a particular student and filter/input snapshot.
Recommended Mentor	A candidate result attached to one recommendation run.
Projection	A graph representation derived from authoritative MySQL data.
Outbox	A durable MySQL event row written in the same transaction as source changes.
Cache-aside	Application checks cache first; on a miss it computes from source and then caches.
Cold start	A profile or mentor with limited matching / outcome history.

24. Source Basis and Document Alignment

This recommendation design is aligned to the supplied project artefacts. Where this document proposes numerical parameters or operational policies that the sources do not specify, they are labelled as recommended defaults and must be confirmed by the project team.

Supplied source	Relevant sections used
Alumni–Student Mentorship Platform: DBMS Project Blueprint	Project vision and roles; Neo4j graph model and recommendation dimensions (§5.1-5.5); Redis cache-aside; recommendation history; release scope.
AlumniConnect Product Requirements Document, AC-PRD-001	FR-05 to FR-10 (skills, interests, capacity, discovery, graph-based recommendations, explanation/history); NFR-04 / NFR-05 (cross-database consistency and performance target).
AlumniConnect Application Architecture	§4.2 module catalogue; §5.1 discovery flow; §6 MySQL discovery history; §7.1 Redis and §7.2-7.3 Neo4j / recommendation dimensions; §8 transactional outbox.
AlumniConnect Conceptual ER Diagram & Domain Model	Recommendation Run and Recommended Mentor entity catalogue (p. 10); relationship catalogue for student-run/candidate/mentor associations (pp. 13-14).
AlumniConnect Workflow and State Diagrams	MySQL source-of-truth boundary, actor and outbox responsibilities; request / allocation workflow and state-history conventions.
DBMS Unit 1-4 course material	Relational design, advanced SQL, normalization / query optimization, transactions, Redis key-value and Neo4j graph topics as reflected by the project blueprint.

Final design statement

AlumniConnect uses graph traversal to find relevant alumni, MySQL to enforce who is currently eligible, a versioned weighted score to rank eligible candidates, evidence-based explanations to build student trust, and persistent recommendation history to evaluate what happened after selection.